

Algorithmic Analysis of Advanced Stack Applications: Monotonic Optimizations and Subarray Bounds

Abstract

This paper provides a formal algorithmic analysis of stack data structures and their advanced applications. We begin by detailing a dynamically allocated linked list implementation for core stack operations, formalizing $O(1)$ constant-time push, pop, and peek functions. We then demonstrate a stack-based approach for adjacent character elimination in strings. Furthermore, we delve into Monotonic Stacks, presenting optimal linear-time $O(N)$ solutions for the Largest Rectangle in a Histogram problem using Next Smaller Element left and right variants. Finally, we formulate an elegant subarray bounds algorithm that leverages monotonic structures to compute the sum of differences between maximum and minimum elements across all possible contiguous subarrays in $O(N)$ time.

Index Terms

linked list, deep copy, random pointer, hash map, node interleaving, Java, algorithm analysis, computational complexity



1 Introduction

Stacks are a fundamental data structure...

2 Linked List Implementation of Stacks

In a linked list implementation, the stack's top is represented by the head of the list. Pushing an element corresponds to inserting a new node at the head, and popping corresponds to removing the head node.

```
void push(int x) {
    Node* nn = new Node(x);
    nn->next = head;
    head = nn;
}

int pop() {
    if (head == NULL) return -1;
    int save = head->data;
    head = head->next;
    return save;
}

int peek() {
    if (head == NULL) return -1;
    return head->data;
}

boolean isEmpty() {
    if (head == NULL) return true;
    else return false;
}
```

The time complexity for all the above stack operations is $O(1)$.

3 Application I: Removing Equal Adjacent Pairs

Given a string of characters, we want to recursively remove pairs of adjacent equal characters (e.g., $abbc \rightarrow ac$). Using a stack, we iterate through each character in the string. If the stack is not empty and the current character matches the character at

the top of the stack, we pop the element. Otherwise, we push the current character. After processing the entire string, the stack contains the final reduced string. This takes $O(N)$ time and $O(N)$ auxiliary space.

4 Application II: Largest Rectangle in a Histogram

Given an integer array A where $A[i]$ represents the height of the i -th bar, and each bar has a width of 1, we want to find the area of the largest rectangle formed by contiguous bars. For example, given $A = [8, 6, 2, 5, 6, 5, 7, 4]$, the maximum area is 20.

4.1 Algorithm using Monotonic Stacks

For every bar i , we assume it forms the minimum height of a potential maximum rectangle. The rectangle can expand to the left until it hits a bar smaller than $A[i]$ (called the Next Smaller to Left, or NSL), and similarly to the right (Next Smaller to Right, or NSR). The area formed by treating $A[i]$ as the minimum height is:

$$\text{Area} = A[i] \times (NSR[i] - NSL[i] - 1) \quad (1)$$

By precomputing the NSL and NSR arrays for all elements using monotonic stacks in $O(N)$ time, the maximum area over all bars can be found in $O(N)$ total time.

5 Application III: Sum of Subarray Ranges (Max - Min)

We want to compute the sum of the difference between the maximum and minimum elements for all subarrays of a given array A . Instead of evaluating all $O(N^2)$ subarrays, we can calculate the total by finding the sum of maximums of all subarrays minus the sum of minimums of all subarrays.

$$\text{Total} = \sum_{i=0}^{N-1} A[i] \times (\# \text{ of subarrays where } A[i] \text{ is max}) - \sum_{i=0}^{N-1} A[i] \times (\# \text{ of subarrays where } A[i] \text{ is min}) \quad (2)$$

5.1 Counting Subarrays

To count how many subarrays have $A[i]$ as their maximum, we find the Next Greater to Left (NGL) and Next Greater to Right (NGR) indices. The number of such subarrays is $(i - NGL) \times (NGR - i)$. Similarly, to count subarrays where $A[i]$ is the minimum, we use Next Smaller to Left (NSL) and Next Smaller to Right (NSR). The algorithm operates in $O(N)$ time:

```
long total = 0;
for (int i = 0; i < N; i++) {
    long max_count = (i - ngl[i]) * (ngr[i] - i);
    long min_count = (i - nsl[i]) * (nsr[i] - i);
    total += (max_count - min_count) * A[i];
}
return total;
```

6 Conclusion

The use of stacks, particularly monotonic stacks, allows us to reduce worst-case $O(N^2)$ time algorithms to linear time $O(N)$ complexity for a variety of subarray optimization and range queries.